Deploying Machine Learning Models: A Comparative Study of Flask, FastAPI, and Streamlit

Siddheshwar Tewari

University of Washington, Seattle

sidtewar@uw.edu

Abstract

Deployment of machine learning (ML) models into production environments is a critical stage of the machine learning lifecycle. A variety of frameworks exist to facilitate serving models as APIs or interactive applications, with Flask, FastAPI, and Streamlit emerging as popular tools among practitioners. This paper presents a comparative study of these three frameworks based on performance, ease of use, scalability, and developer experience. Through systematic benchmarking and qualitative analysis, I highlight their trade-offs and provide recommendations for practitioners seeking efficient and scalable ML deployment solutions.

1 Introduction

As machine learning has matured from research prototypes to real-world applications, model deployment has become an essential step for bridging the gap between data science teams and production systems as per [5]Sculley David. Without deployment, models remain static artifacts, disconnected from the environments where they can generate value. Effective deployment requires not only exposing models via APIs or user interfaces, but also ensuring maintainability, reproducibility, and scalability.

[1]Grinberg, an author specializing in ML model deployments stated that Flask, FastAPI, and Streamlit have gained traction as lightweight yet powerful frameworks for ML model serving. Flask is a micro web framework offering flexibility and extensibility. FastAPI, a modern Python framework, emphasizes high performance with asynchronous support as per the documentation of [4]FastAPI. Streamlit, in contrast, focuses on rapid prototyping and interactive visualization for data scientists without requiring web development expertise as stated in the [2]Streamlit FAQ page.

This study aims to compare these frameworks systematically to provide practical guidance for data scientists, ML engineers, and researchers making deployment decisions.

2 Related Work

Model deployment is closely tied to the discipline of Machine Learning Operations (MLOps), which integrates DevOps principles with the ML lifecycle according to [3]Kreuzberger. The growing emphasis on reproducibility and continuous integration has led to significant research on containerization, orchestration, and monitoring. However, the choice of serving framework is often treated as a secondary concern, left to individual developer preference.

Prior studies have examined performance differences among web frameworks in general contexts, but fewer works specifically investigate how such frameworks interact with machine learning workflows. Moreover, while industrial white papers discuss best practices in ML deployment, empirical comparisons between lightweight frameworks such as Flask, FastAPI, and Streamlit remain scarce. This paper seeks to fill that gap by providing systematic benchmarks and a structured analysis of trade-offs.

3 Methodology

I evaluate Flask, FastAPI, and Streamlit across four criteria that reflect common requirements for ML deployment:

1. **Performance:** Response time and throughput under concurrent requests.
2. **Ease of Use:** Learning curve, documentation quality, and required code complexity.

3. **Scalability:** Ability to handle production-grade workloads, particularly under high concurrency.

4. **Developer Experience:** Debugging, integration with ML libraries, and deployment workflow simplicity.

A simple ML model (logistic regression for text classification) is deployed in each framework. To ensure fairness, identical pre-trained models are used, and API endpoints are designed with equivalent input/output formats. Performance is tested using Apache Benchmark with varying levels of concurrent requests.

Qualitative evaluation was also conducted. I recorded developer observations on time to initial deployment, difficulty of integrating the model pipeline, and flexibility for adding monitoring or logging features.

4 Experimental Setup

All experiments were conducted on a machine with 16GB RAM, Intel i7 processor, and Ubuntu 22.04. Each framework was configured with default settings. Flask applications were served with Gunicorn, FastAPI with Uvicorn, and Streamlit using its built-in server. Performance was evaluated under 50, 100, and 500 concurrent requests, simulating small-scale to moderate workloads.

In addition, ease of use was assessed by measuring the number of lines of code required for minimal deployment. Documentation quality was evaluated based on availability of ML-specific tutorials, examples, and community support.

5 Results and Discussion

5.1 Performance

My findings indicate that FastAPI consistently outperforms Flask and Streamlit in terms of latency and throughput due to its asynchronous capabilities. Under 500 concurrent requests, FastAPI maintained sub-50ms response times, while Flask's response times increased substantially. Streamlit was unsuitable for concurrent API-style workloads, as its design emphasizes interactive dashboards rather than high-throughput endpoints.

5.2 Ease of Use

In terms of ease of use, Streamlit emerged as the most beginner-friendly option. A model could be deployed with fewer than 20 lines of Python code, and no knowledge of HTML or JavaScript was required. Flask and FastAPI required more boilerplate, though FastAPI's automatic documentation via OpenAPI provided a smoother developer experience for building APIs.

5.3 Scalability

Flask has been widely adopted in production systems and benefits from a mature ecosystem of extensions. However, it lacks native asynchronous support, limiting scalability in high-concurrency scenarios. FastAPI addresses this gap with built-in async functionality, making it highly scalable for real-time ML inference

services. Streamlit is better suited for small-scale applications, rapid prototyping, and demos, rather than heavy production workloads.

5.4 Developer Experience

Flask offers maximal flexibility but requires manual integration of libraries for monitoring, authentication, and testing. FastAPI balances flexibility with developer convenience, providing type hints, validation, and built-in async support. Streamlit sacrifices flexibility in favor of accessibility, enabling non-engineers to build interactive interfaces with minimal code.

6 Conclusion and Future Work

This paper has presented a comparative analysis of Flask, FastAPI, and Streamlit for ML model deployment. While FastAPI emerges as the most performant option, the choice of framework ultimately depends on the specific use case and developer expertise. Flask remains a reliable general-purpose option for web applications, whereas Streamlit excels in rapid prototyping and visualization.

Future work will expand benchmarking to include containerized deployments with Docker, orchestration systems such as Kubernetes, and real-world case studies on cloud platforms. Additionally, further research could incorporate advanced monitoring and explainability tools to assess how well each framework integrates with modern MLOps practices.

References

[1] Miguel Grinberg. *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, 2018.

[2] Streamlit Inc. Streamlit documentation. `https://docs.streamlit.io/`, 2022.

[3] Dominik Kreuzberger, Niklas Kühl, and Sven Hirschl. Machine learning operations (mlops): Overview, definition, and architecture. *ACM Computing Surveys*, 2023.

[4] Sebastián Ramirez. Fastapi documentation. `https://fastapi.tiangolo.com/`, 2023.

[5] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.